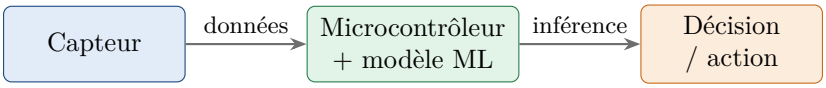
Module : Internet des Objets

L'AIoT & le TinyML

Quand l'intelligence artificielle s'intègre dans les objets connectés



Rapport de veille documentaire

Réalisé par :

Prénom NOM · Prénom NOM · Prénom NOM

Année universitaire 2026–2027

Résumé

L'Internet des objets (IoT) a permis de connecter des milliards de capteurs et d'actionneurs au réseau. Pendant longtemps, ces objets se sont contentés de mesurer puis de transmettre : l'analyse des données et la prise de décision étaient confiées à des serveurs distants, dans le cloud. L'*Artificial Intelligence of Things* (AIoT) désigne la convergence de l'IoT et de l'intelligence artificielle, et le *Tiny Machine Learning* (TinyML) en constitue la forme la plus contrainte : l'exécution de modèles d'apprentissage automatique directement sur des microcontrôleurs disposant de quelques centaines de kilooctets de mémoire et consommant quelques milliwatts.

Ce rapport présente d'abord les notions d'IoT, d'AIoT et de TinyML, ainsi que l'architecture qui les relie. Il détaille ensuite les principes techniques qui rendent le TinyML possible (pipeline de développement, quantification, élagage, distillation), le matériel et les outils logiciels disponibles, puis les bénéfices, les domaines d'application, les limites et les perspectives de cette approche. Le fil conducteur est un cas d'usage concret : la détection d'anomalies de vibration sur un moteur à l'aide d'un ESP32.

Mots-clés : Internet des objets, AIoT, TinyML, Edge AI, microcontrôleur, ESP32, quantification, inférence embarquée.

Table des matières

Résumé **i**

Introduction générale **1**

1 De l'IoT à l'AIoT et au TinyML **2**
1.1 L'Internet des objets . . . 2
1.1.1 Qu'est-ce qu'un objet connecté ? . . . 2
1.1.2 Architecture générale d'un système IoT . . . 2
1.1.3 Les limites du modèle « tout cloud » . . . 3
1.2 L'AIoT : l'intelligence artificielle au service des objets . . . 4
1.2.1 Définition . . . 4
1.2.2 Pourquoi ajouter de l'IA aux objets connectés ? . . . 4
1.2.3 Exemples d'utilisation . . . 4
1.3 Le TinyML : l'apprentissage automatique sur microcontrôleur . . . 4
1.3.1 Définition . . . 5
1.3.2 Les contraintes du TinyML . . . 5
1.3.3 Entraînement et inférence : une séparation fondamentale . . . 6
1.4 Fonctionnement d'une architecture AIoT/TinyML . . . 6
1.4.1 La chaîne capteur – microcontrôleur – modèle – action . . . 6
1.4.2 Comparaison avec une architecture centrée sur le cloud . . . 7
1.4.3 Le cas d'étude retenu . . . 8

2 Principes techniques du TinyML **9**
2.1 Introduction . . . 9
2.2 Pipeline de développement TinyML . . . 9
2.3 Techniques de compression et d'optimisation des modèles . . . 10
2.3.1 Introduction à la compression . . . 10
2.3.2 La quantification (*quantization*) . . . 10
2.3.3 L'élagage (*pruning*) . . . 13
2.3.4 La distillation (*knowledge distillation*) . . . 14
2.3.5 Combinaison des techniques . . . 15
2.3.6 Techniques avancées . . . 15
2.3.7 Métriques d'évaluation . . . 15
2.3.8 Études de cas . . . 16

3 Matériel et outils de développement **17**
3.1 Architectures matérielles . . . 17
3.1.1 ESP32-S3 (Espressif) . . . 17
3.1.2 ESP32-P4 (Espressif) . . . 17
3.1.3 STM32N6 (STMicroelectronics) . . . 17
3.2 Chaîne d'outils logiciels . . . 18
3.2.1 TensorFlow Lite for Microcontrollers (TFLM) . . . 18
3.2.2 Edge Impulse . . . 18
3.2.3 Outils spécifiques aux fabricants . . . 19

4 Bénéfices et domaines d'application **20**
4.1 Les bénéfices de l'AIoT et du TinyML . . . 20
4.1.1 Une meilleure réactivité et une moindre dépendance au réseau . . . 20
4.1.2 Une meilleure maîtrise de la confidentialité des données . . . 20

4.1.3 Une réduction possible de la consommation énergétique 20
4.2 Domaines d'application et exemples d'utilisation 21
4.2.1 Industrie : surveillance des machines et maintenance 21
4.2.2 Maison connectée : reconnaissance de mots-clés et de sons 21

5 Limites, défis et perspectives d'avenir 22
5.1 Des ressources matérielles limitées 22
5.2 Une fiabilité dépendante des données et du contexte 22
5.3 L'entraînement et l'évolution des modèles 22
5.4 La sécurité des modèles embarqués 23
5.5 Vers des architectures complémentaires entre l'objet et le cloud 23

Conclusion générale 24

Bibliographie et webographie 25

Introduction générale

En une quinzaine d'années, l'Internet des objets (*Internet of Things*, IoT) est passé du statut de concept à celui d'infrastructure du quotidien : thermostats, montres, compteurs électriques, machines industrielles ou capteurs agricoles sont désormais connectés. Le cabinet IoT Analytics estimait à environ 18,8 milliards le nombre d'objets connectés actifs dans le monde fin 2024 [23]. Ces objets produisent un volume de données considérable, que l'architecture « classique » de l'IoT envoie vers le cloud pour y être stockées, analysées et transformées en décisions.

Ce modèle centralisé montre aujourd'hui ses limites. Transmettre en permanence des mesures brutes consomme de la bande passante et de l'énergie, introduit une latence qui dépend de la qualité du réseau, rend le système inopérant lorsque la connexion est coupée et expose des données parfois sensibles (sons, images, mesures physiologiques). Le paradigme de l'*edge computing* propose de rapprocher le traitement de la source des données [17].

L'étape suivante consiste à doter les objets eux-mêmes d'une capacité d'analyse : c'est l'**AIoT**, contraction d'*Artificial Intelligence* et d'*Internet of Things* [18]. Lorsque cette intelligence est logée dans les composants les plus modestes de la chaîne, des microcontrôleurs de quelques euros fonctionnant sur pile, on parle de **TinyML** [2, 5]. Un capteur de vibrations ne transmet plus des milliers d'échantillons par seconde : il signale simplement « fonctionnement anormal détecté ».

Problématique

Comment faire fonctionner des modèles d'intelligence artificielle, conçus habituellement pour des serveurs dotés de gigaoctets de mémoire, sur des objets connectés qui ne disposent que de quelques centaines de kilooctets, et quels bénéfices et limites cette approche apporte-t-elle aux systèmes IoT ?

Pour y répondre, ce rapport est organisé en cinq parties. La **partie 1** pose les bases : objet connecté, architecture d'un système IoT, définition de l'AIoT et du TinyML, et comparaison entre une architecture centrée sur le cloud et une architecture où l'inférence est réalisée localement. La **partie 2** détaille les principes techniques du TinyML, en particulier le pipeline de développement et les techniques de compression des modèles. La **partie 3** présente le matériel et la chaîne d'outils logiciels. La **partie 4** examine les bénéfices de l'approche et ses domaines d'application. Enfin, la **partie 5** discute les limites, les défis et les perspectives d'avenir.

Partie 1

De l'IoT à l'AIoT et au TinyML

Avant d'étudier la manière dont un modèle d'intelligence artificielle peut être exécuté sur un microcontrôleur, il est nécessaire de préciser le vocabulaire et le cadre dans lequel s'inscrit ce travail. Cette partie définit successivement l'objet connecté et l'architecture d'un système IoT (section 1.1), l'AIoT (section 1.2), le TinyML (section 1.3), puis compare les architectures centrées sur le cloud à celles où l'inférence est réalisée dans l'objet (section 1.4).

1.1 L'Internet des objets

1.1.1 Qu'est-ce qu'un objet connecté ?

L'expression *Internet of Things* est attribuée à Kevin Ashton, qui l'utilise dès 1999 pour décrire des objets capables de collecter eux-mêmes des informations sur le monde physique, sans saisie humaine [19]. La recommandation ITU-T Y.2060 de l'Union internationale des télécommunications en donne une définition de référence : l'IoT est « une infrastructure mondiale pour la société de l'information, qui permet de disposer de services évolués en interconnectant des objets (physiques ou virtuels) grâce aux technologies de l'information et de la communication interopérables existantes ou en évolution » [25].

Un **objet connecté** est donc un dispositif physique qui associe :

— un ou plusieurs **capteurs** qui mesurent une grandeur physique (température, accélération, son, lumière, humidité, pression…) ;

— une **unité de traitement**, le plus souvent un microcontrôleur (MCU), qui pilote les capteurs et met en forme les données ;

— un **module de communication** (Wi-Fi, Bluetooth Low Energy, LoRaWAN, NB-IoT, Zigbee…) qui relie l'objet au réseau ;

— éventuellement des **actionneurs** (relais, moteur, LED, buzzer) qui agissent sur l'environnement ;

— une **source d'énergie**, souvent une batterie ou une pile, parfois complétée par de la récupération d'énergie (panneau solaire, vibrations).

La carte ESP32 d'Espressif illustre bien ce type de plateforme : un microcontrôleur cadencé à 240 MHz, environ 520 Ko de SRAM, du Wi-Fi et du Bluetooth intégrés, pour un coût de quelques euros [20]. La contrainte majeure de ces objets n'est pas tant leur puissance de calcul que leur **budget énergétique** : un capteur installé dans un champ ou sur une machine doit souvent fonctionner plusieurs mois, voire plusieurs années, sans intervention.

1.1.2 Architecture générale d'un système IoT

Il n'existe pas d'architecture IoT unique, mais la littérature converge vers un découpage en couches [7, 3]. Le modèle le plus courant distingue quatre niveaux, représentés à la figure 1.1 :

1. **La couche perception** (ou couche objets) regroupe les capteurs et actionneurs. Elle convertit des phénomènes physiques en signaux numériques et, inversement, exécute des commandes.

2. **La couche réseau** (ou transport) achemine les données. Elle s'appuie sur des protocoles radio adaptés aux faibles débits et à la faible consommation (BLE, Zigbee, LoRaWAN, NB-IoT)

ainsi que sur des passerelles (*gateways*) qui relaient les messages vers Internet.

3. **La couche traitement** (ou plateforme, *middleware*) stocke, agrège et analyse les données, le plus souvent dans le cloud. On y trouve les bases de données, les tableaux de bord et, aujourd'hui, les modèles d'apprentissage automatique.

4. **La couche application** fournit le service final à l'utilisateur : application mobile, alerte de maintenance, pilotage d'un système d'irrigation, etc.

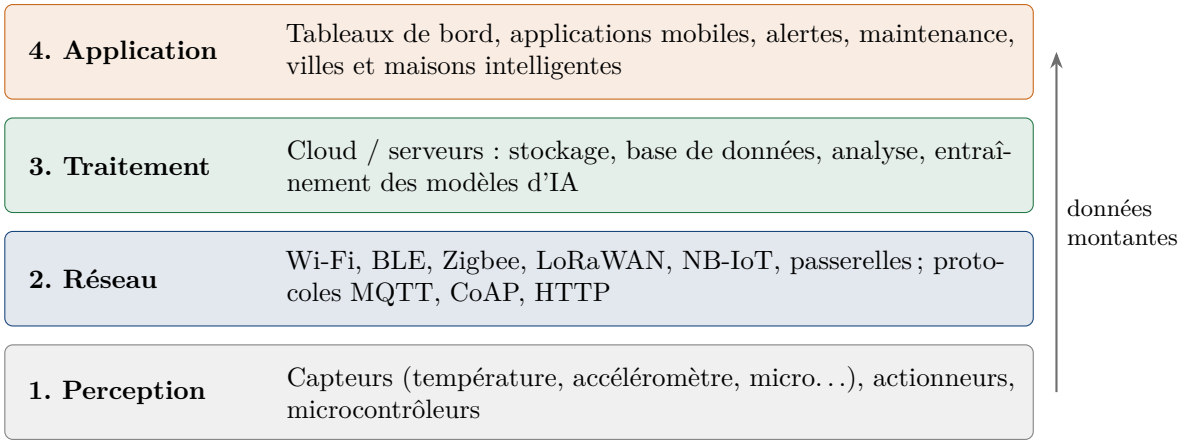


Figure 1.1 – Architecture en quatre couches d'un système IoT (d'après [7]).

Dans ce schéma classique, l'« intelligence » du système réside dans la couche traitement. L'objet est un simple producteur de données : il mesure, transmet et attend éventuellement un ordre en retour. Cette organisation a l'avantage de la simplicité et de la puissance (le cloud dispose de ressources pratiquement illimitées), mais elle fait dépendre toute décision de la disponibilité et de la qualité du réseau.

1.1.3 Les limites du modèle « tout cloud »

Plusieurs difficultés apparaissent lorsque le nombre d'objets et le volume de données augmentent [17] :

— **La bande passante** : un accéléromètre échantillonné à 1 kHz sur trois axes en 16 bits produit environ 6 Ko/s, soit plus de 500 Mo par jour et par capteur. Multiplier ce flux par des centaines de machines sature rapidement un réseau LoRaWAN ou NB-IoT, dont les débits se comptent en kilobits par seconde.

— **La latence** : l'aller-retour vers un serveur ajoute des délais variables, parfois de plusieurs centaines de millisecondes, incompatibles avec certaines réactions (arrêt d'une machine, détection d'une chute).

— **L'énergie** : sur un objet sur batterie, la transmission radio est souvent le poste de consommation le plus important, bien devant le calcul [2].

— **La confidentialité** : envoyer en continu le son d'une pièce ou le rythme cardiaque d'une personne multiplie les risques de fuite.

— **La disponibilité** : sans connexion, l'objet devient « aveugle ».

Ces limites motivent le déplacement d'une partie de l'analyse vers la périphérie du réseau (*edge*), puis vers l'objet lui-même.

1.2 L'AIoT : l'intelligence artificielle au service des objets

1.2.1 Définition

L'**AIoT** (*Artificial Intelligence of Things*) désigne la combinaison des technologies de l'intelligence artificielle, en particulier de l'apprentissage automatique, avec l'infrastructure de l'Internet des objets [18]. L'objectif est de transformer les données collectées par les objets en informations exploitables (reconnaître une situation, détecter une anomalie, anticiper un événement) et, idéalement, de déclencher une action sans intervention humaine.

L'IA peut être placée à différents endroits de l'architecture : dans le cloud, sur une passerelle ou un serveur de proximité (*edge server*), ou directement dans l'objet (*on-device AI*). L'AIoT recouvre l'ensemble de ces configurations ; ce qui la caractérise est le passage d'objets qui *mesurent* à des systèmes qui *comprennent* et *décident*.

1.2.2 Pourquoi ajouter de l'IA aux objets connectés ?

Les règles fixes (« si la température dépasse 80 °C, alors alarme ») suffisent pour des phénomènes simples. Elles deviennent inadaptées lorsque le signal est complexe : reconnaître un mot prononcé, distinguer le bruit normal d'un roulement de celui d'un roulement usé, identifier un geste à partir d'un accéléromètre. Dans ces cas, il est très difficile d'écrire les règles à la main, alors qu'un modèle d'apprentissage automatique peut les apprendre à partir d'exemples. L'IA apporte ainsi aux systèmes IoT :

— la **reconnaissance de motifs** dans des signaux bruités (audio, vibrations, images) ;

— la **détection d'anomalies**, c'est-à-dire l'identification d'écarts par rapport à un comportement appris comme normal ;

— la **prédiction** de l'évolution d'une grandeur (consommation, usure, rendement) ;

— l'**adaptation** au contexte et à l'utilisateur.

1.2.3 Exemples d'utilisation

Domaine	Données collectées	Apport de l'IA
Industrie	Vibrations, courant, température des moteurs	Détection d'anomalies, maintenance prédictive
Maison	Son, présence, consommation électrique	Reconnaissance de mots-clés, automatisation
Agriculture	Humidité du sol, météo, images de feuilles	Irrigation optimisée, détection de maladies
Santé	Fréquence cardiaque, mouvements	Détection de chutes ou d'arythmies
Transport	Accélérations, images, données moteur	Aide à la conduite, surveillance du véhicule
Environnement	Sons de la forêt, qualité de l'air	Détection de tronçonneuses, alerte pollution

Table 1.1 – Exemples d'applications de l'AIoT.

Ces exemples, développés dans la partie 4, ont un point commun : la décision utile (« anomalie », « mot-clé reconnu », « chute ») représente une quantité d'information infime par rapport aux données brutes dont elle est issue. C'est précisément ce qui rend intéressante l'exécution du modèle au plus près du capteur.

1.3 Le TinyML : l'apprentissage automatique sur microcontrôleur

1.3.1 Définition

Le **TinyML** (*Tiny Machine Learning*) désigne le domaine qui regroupe les technologies, matérielles et logicielles, permettant d'exécuter des modèles d'apprentissage automatique sur des appareils à très faibles ressources, typiquement des microcontrôleurs, avec une consommation de l'ordre du milliwatt ou moins [2, 5]. Le terme a été popularisé en 2019 par l'ouvrage de Pete Warden et Daniel Situnayake [2] et par la création de la communauté TinyML Foundation, devenue depuis EDGE AI FOUNDATION [26].

Le TinyML se situe à l'extrémité du continuum de l'IA embarquée, comme le montre le tableau 1.2. Là où un serveur dispose de dizaines de gigaoctets de mémoire et un smartphone de plusieurs gigaoctets, un microcontrôleur n'en possède que quelques centaines de kilooctets, soit un facteur de l'ordre de 10^5.

	Cloud AI	**Mobile / Edge AI**	**TinyML**
Matériel	Serveurs GPU/TPU	Smartphone, Raspberry Pi, Jetson	Microcontrôleur (ESP32, Arm Cortex-M)
Mémoire vive	Dizaines à centaines de Go	1 à 16 Go	32 Ko à quelques Mo
Stockage	To	Dizaines de Go	256 Ko à quelques Mo de flash
Puissance	Centaines de watts	Quelques watts	Milliwatts
Système	Linux, conteneurs	Android, Linux	Pas d'OS ou RTOS (FreeRTOS)
Coût unitaire	Milliers d'euros	50 à 500 €	1 à 10 €
Taille de modèle	Go	Dizaines de Mo	Ko à quelques centaines de Ko

Table 1.2 – Positionnement du TinyML par rapport aux autres formes d'IA (ordres de grandeur, d'après [5, 14]).

1.3.2 Les contraintes du TinyML

Concevoir une application TinyML revient à respecter simultanément plusieurs contraintes très fortes [5, 16] :

La mémoire. Un microcontrôleur dispose de deux types de mémoire. La **mémoire flash** (non volatile) stocke le programme et les poids du modèle ; la **SRAM** (volatile) contient les données de travail, en particulier les activations intermédiaires calculées pendant l'inférence. Les deux sont limitées : un modèle doit typiquement tenir dans quelques centaines de kilooctets de flash et ses activations dans quelques dizaines à centaines de kilooctets de SRAM. Lin *et al.* montrent que c'est souvent la SRAM, plus encore que la flash, qui constitue le goulot d'étranglement [14].

La puissance de calcul. Les microcontrôleurs fonctionnent à des fréquences de quelques dizaines à quelques centaines de mégahertz, souvent sans unité de calcul flottant performante. Un réseau de neurones de quelques millions d'opérations est envisageable ; un modèle de plusieurs milliards d'opérations ne l'est pas.

L'énergie. Pour fonctionner des mois sur une pile, l'objet doit passer l'essentiel de son temps en veille et limiter la durée de chaque inférence. La consommation se mesure alors en milliwatts et l'énergie par inférence en microjoules ou millijoules [4].

L'environnement logiciel. Pas de système d'exploitation complet, pas toujours d'allocation dynamique de mémoire, pas de Python : le modèle est compilé avec le micrologiciel (*firmware*) en C/C++ et exécuté par un moteur d'inférence minimaliste comme TensorFlow Lite for Microcontrollers (TFLM) [6].

1.3.3 Entraînement et inférence : une séparation fondamentale

Il est essentiel de distinguer les deux phases de vie d'un modèle :

— l'**entraînement**, qui consiste à ajuster les paramètres du modèle à partir d'un grand nombre d'exemples. Il demande beaucoup de mémoire et de calcul et s'effectue sur un ordinateur ou dans le cloud ;

— l'**inférence**, qui consiste à appliquer le modèle déjà entraîné à une nouvelle donnée pour obtenir une prédiction. C'est la seule phase exécutée sur le microcontrôleur dans la grande majorité des systèmes TinyML [6].

Entre les deux, le modèle doit être **optimisé** (réduit, quantifié) puis **converti** dans un format exécutable par le microcontrôleur. Ces étapes sont détaillées dans la partie 2.

À retenir

L'IoT connecte les objets, l'AIoT leur donne la capacité d'interpréter leurs données, et le TinyML pousse cette logique jusqu'aux plus petits composants : des microcontrôleurs à quelques euros, alimentés sur pile, qui exécutent localement un modèle entraîné ailleurs.

1.4 Fonctionnement d'une architecture AIoT/TinyML

1.4.1 La chaîne capteur – microcontrôleur – modèle – action

Dans un système TinyML, le traitement suit une boucle locale composée de cinq étapes (figure 1.2) :

1. **Acquisition** : le capteur mesure le phénomène (par exemple l'accélération d'un moteur) et le microcontrôleur lit les échantillons via un bus I^2C, SPI ou un convertisseur analogique-numérique.
2. **Prétraitement** : les échantillons sont regroupés dans une fenêtre temporelle (par exemple une seconde) puis transformés en caractéristiques plus compactes : moyenne, écart-type, valeur efficace, spectre par transformée de Fourier (FFT), coefficients MFCC pour l'audio.
3. **Inférence** : le vecteur de caractéristiques est fourni au modèle, qui renvoie une prédiction (classe, score d'anomalie, valeur).
4. **Décision** : la prédiction est comparée à un seuil ou interprétée par une règle simple.
5. **Action et communication** : l'objet agit (LED, relais, arrêt d'urgence) et ne transmet au réseau que l'information utile, par exemple un message MQTT « anomalie détectée ».

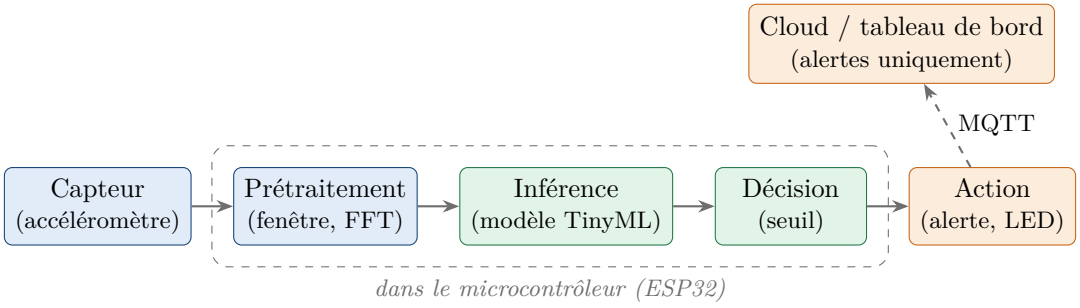


Figure 1.2 – Boucle de traitement locale d'un objet AIoT/TinyML.

Le listing 1.1 montre, sous forme simplifiée, la structure du programme principal d'un ESP32 qui applique ce principe. Le modèle est interrogé à chaque fenêtre de mesures et seule une alerte est publiée.

Listing 1.1 – Structure simplifiée de la boucle d'inférence sur ESP32.

```
void loop() {
  // 1. Acquisition : une fenêtre de 1 s à 1 kHz sur 3 axes
  lire_accelerometre(fenetre, 1000);

  // 2. Prétraitement : extraction de caractéristiques (RMS, FFT...)
  calculer_caracteristiques(fenetre, entree_modele);

  // 3. Inférence locale (TensorFlow Lite for Microcontrollers)
  interpreter->Invoke();
  float score = sortie_modele[0];

  // 4-5. Décision et action : on ne transmet que l'événement utile
  if (score > SEUIL_ANOMALIE) {
    digitalWrite(LED_ALERTE, HIGH);
    mqtt.publish("usine/moteur1/alerte", "anomalie");
  }
  esp_light_sleep_start();   // mise en veille jusqu'à la prochaine fenêtre
}
```

1.4.2 Comparaison avec une architecture centrée sur le cloud

Dans une architecture IoT classique, l'objet transmet en continu ses mesures brutes ; la passerelle les relaie vers le cloud, où le modèle est exécuté avant que la décision ne redescende éventuellement vers l'objet. Dans une architecture TinyML, l'inférence est réalisée dans l'objet et seules les décisions remontent. La figure 1.3 compare les deux flux.

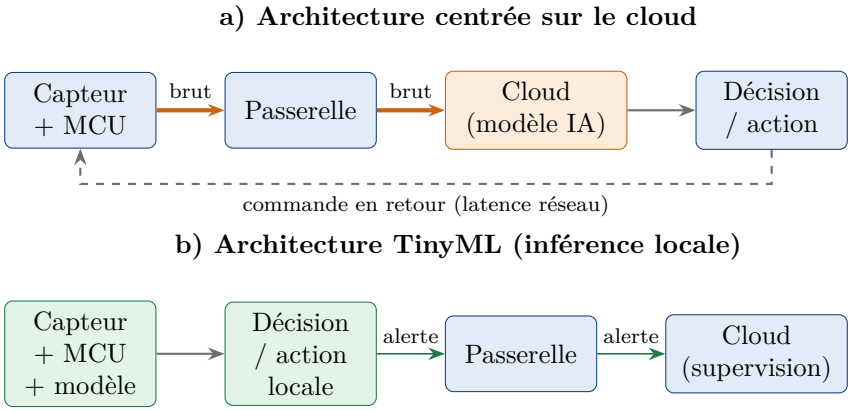


Figure 1.3 – Flux de données : architecture cloud (données brutes, flèches épaisses) et architecture TinyML (seules les alertes remontent).

Critère	Architecture cloud	Architecture TinyML
Lieu de l'inférence	Serveur distant	Microcontrôleur de l'objet
Données transmises	Mesures brutes, en continu	Résultats et alertes uniquement
Latence de décision	Dépend du réseau (de dizaines de ms à plusieurs secondes)	Quelques ms à quelques dizaines de ms, indépendante du réseau
Fonctionnement hors ligne	Impossible	Possible pour les fonctions locales
Confidentialité	Données sensibles transférées et stockées à distance	Données brutes conservées dans l'objet
Complexité des modèles	Pratiquement illimitée	Fortement contrainte (Ko de mémoire)
Mise à jour du modèle	Simple (côté serveur)	Plus délicate (mise à jour du firmware, OTA)
Coût récurrent	Abonnement cloud, transfert de données	Faible, surtout lié à la supervision

Table 1.3 – Comparaison entre une architecture centrée sur le cloud et une architecture TinyML.

Les deux approches ne s'opposent pas nécessairement. Dans la pratique, les architectures AIoT sont souvent **hybrides** : l'objet filtre et interprète localement les données, une passerelle ou le cloud agrège les résultats de nombreux objets, réalise les analyses lourdes et ré-entraîne périodiquement les modèles, qui sont ensuite redéployés dans les objets. Cette complémentarité est discutée dans la partie 5.

1.4.3 Le cas d'étude retenu

Pour illustrer concrètement ces principes, ce rapport s'appuie sur un exemple fil rouge : la **détection d'anomalies vibratoires sur un moteur**. Un accéléromètre (de type MPU6050 ou ADXL345) est fixé sur le carter du moteur et relié à un ESP32. Un petit modèle, entraîné sur ordinateur à partir de mesures en fonctionnement normal, est quantifié puis intégré au firmware. L'ESP32 analyse chaque seconde de vibrations et publie une alerte lorsque le comportement s'écarte de la normale. Ce scénario mobilise toutes les briques étudiées dans les parties suivantes : pipeline de développement, compression du modèle, matériel et outils, bénéfices et limites.

Partie 2

Principes techniques du TinyML

2.1 Introduction

La partie précédente a permis de définir le concept de TinyML et d'en identifier les enjeux fondamentaux. Cette partie examine les mécanismes techniques sous-jacents rendant possible l'exécution de modèles d'intelligence artificielle sur des dispositifs embarqués aux ressources limitées.

La problématique centrale s'énonce ainsi : comment déployer un modèle d'apprentissage automatique, dont la taille peut atteindre plusieurs centaines de mégaoctets, sur un microcontrôleur disposant seulement de quelques centaines de kilooctets de mémoire vive ? La réponse repose sur trois piliers : un pipeline de développement rigoureusement structuré, l'application de techniques avancées de compression des modèles, et une exécution optimisée sur le matériel cible.

2.2 Pipeline de développement TinyML

Le TinyML ne consiste pas en l'entraînement direct des modèles sur le microcontrôleur. Le dispositif embarqué n'effectue qu'une seule opération : l'**inférence**, c'est-à-dire l'exécution d'un modèle préalablement entraîné et optimisé. L'intégralité des calculs intensifs s'effectue en amont, sur des stations de travail ou dans le cloud [2].

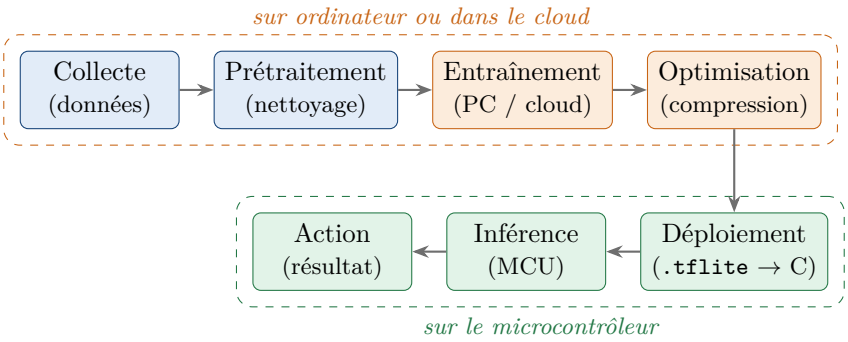


Figure 2.1 – Pipeline complet de développement TinyML.

Le processus se décompose en cinq étapes séquentielles :

1. **Collecte et préparation des données** : l'acquisition de données représentatives constitue l'étape fondatrice. Les données proviennent généralement des capteurs du dispositif final. Cette phase représente une part majeure du temps total de développement, car la qualité d'un modèle est directement dépendante de la qualité des données utilisées.

2. **Entraînement du modèle** : cette phase s'effectue sur des machines dotées de ressources computationnelles importantes, utilisant des frameworks tels que TensorFlow ou PyTorch. Le modèle produit est généralement trop volumineux (plusieurs mégaoctets en Float32) pour les contraintes des microcontrôleurs.

3. **Optimisation du modèle** : étape spécifique et critique du TinyML, elle applique des techniques de compression (quantification, élagage, distillation) pour réduire la taille tout en préservant les performances. Le modèle est converti dans le format FlatBuffer de TensorFlow Lite (`.tflite`).

4. **Intégration dans le firmware** : le modèle optimisé est intégré au code embarqué sous forme de tableau d'octets statique stocké dans la mémoire flash.

5. **Exécution de l'inférence** : le microcontrôleur exécute le modèle sur les données acquises par ses capteurs, sans connexion réseau. Les résultats déclenchent des actions locales.

Caractéristique fondamentale : le modèle est figé au moment de la compilation. Le microcontrôleur n'effectue aucune forme d'apprentissage ; il applique les calculs définis.

2.3 Techniques de compression et d'optimisation des modèles

2.3.1 Introduction à la compression

La compression des modèles représente un domaine essentiel pour le déploiement de l'IA sur dispositifs embarqués [1]. L'objectif consiste à réduire significativement la taille des modèles tout en minimisant la dégradation des performances. Cette réduction permet de respecter les contraintes mémorielles, d'améliorer l'efficacité énergétique et la vitesse d'exécution.

Les techniques peuvent être classées en plusieurs catégories : quantification, réduction de complexité structurelle (élagage, distillation), factorisation et approches hybrides [9].

2.3.2 La quantification (*quantization*)

2.3.2.1 Définition et principes

La quantification est la technique de compression la plus répandue et la plus efficace en TinyML [12, 8]. Elle réduit la précision des représentations numériques des poids et des activations du réseau.

Contexte théorique : en apprentissage classique, les réseaux manipulent des valeurs en virgule flottante 32 bits (Float32), offrant une grande précision mais au prix d'un espace mémoire important (4 octets par nombre). Dans l'embarqué, cette précision est généralement superflue.

Principe : la quantification consiste à représenter les valeurs sur 8 bits entiers (Int8), divisant par quatre la taille mémoire requise. Un modèle de 4 Mo en Float32 n'occupera plus que 1 Mo en Int8.

2.3.2.2 Mécanismes

La quantification implique un processus de mise en correspondance (*mapping*) préservant l'information. Avant d'expliciter ce mécanisme, il est nécessaire de définir ce qu'est un **tenseur**.

Qu'est-ce qu'un tenseur ? En apprentissage automatique, un tenseur est une généralisation des matrices et des vecteurs à un nombre quelconque de dimensions. C'est la structure de données fondamentale utilisée pour représenter les poids, les activations et les données d'entrée dans les réseaux de neurones. Un tenseur d'ordre n est un tableau à n indices ; chaque élément est identifié par n coordonnées $(i_1, i_2, \ldots, i_n)$.

— **Ordre 0** (scalaire) : une simple valeur numérique, par exemple 3,14 ;

— **Ordre 1** (vecteur) : un tableau 1D, par exemple les biais d'une couche : [0,2; −0,5; 1,3; 0,8] ;

— **Ordre 2** (matrice) : un tableau 2D, comme les poids d'une couche entièrement connectée ;

— **Ordre 3** : un cube de données, typiquement les activations d'une couche convolutionnelle avec plusieurs canaux ;

— **Ordre 4** : les poids des couches convolutionnelles (hauteur × largeur × canaux d'entrée × canaux de sortie).

Dans le contexte du TinyML, les **poids** du réseau sont stockés sous forme de tenseurs (généralement d'ordre 2 ou 4), les **activations** (sorties intermédiaires de chaque couche) sont également des tenseurs, et la **quantification** s'applique à chaque tenseur individuellement, avec ses propres paramètres (*scale* et *zero_point*).

Formulation mathématique. La relation fondamentale de la quantification affine s'exprime par [12] :

$$r \approx S \times (q - Z) \tag{2.1}$$

où r est la valeur réelle (Float32), q la valeur quantifiée (entier Int8 dans $[-128, 127]$), S le facteur d'échelle (*scale*, réel positif) et Z le point zéro (*zero_point*, entier de même type que q).

Processus détaillé.

1. **Analyse de la plage dynamique** : pour chaque tenseur, on détermine les valeurs minimale et maximale ;
2. **Calcul des paramètres** : on calcule S et Z pour faire correspondre la plage du tenseur à $[-128, 127]$;
3. **Application de la quantification** : chaque valeur est convertie selon la formule (2.1) inversée ;
4. **Exécution en arithmétique entière** : pendant l'inférence, les multiplications se font en Int8 et les accumulations en Int32.

Exemple détaillé. Considérons un tenseur de poids d'une couche dense avec les valeurs suivantes (en Float32) :

$$W = [0{,}23;\ -0{,}45;\ 0{,}12;\ -0{,}87;\ 0{,}65;\ -0{,}33]$$

Étape 1 – Détermination de la plage : $\min = -0{,}87$, $\max = 0{,}65$, d'où plage $= 0{,}65 - (-0{,}87) = 1{,}52$.

Étape 2 – Calcul du facteur d'échelle : sur 8 bits, l'intervalle $[-128, 127]$ compte 255 pas :

$$S = \frac{\text{plage}}{255} = \frac{1{,}52}{255} \approx 0{,}0059608$$

Étape 3 – Calcul du point zéro : on choisit Z de sorte que la valeur minimale corresponde à -128 :

$$Z = -128 - \text{round}\left(\frac{\min}{S}\right) = -128 - \text{round}(-145{,}95) = -128 + 146 = 18$$

Étape 4 – Quantification de chaque valeur :

$$q_i = \text{round}\left(\frac{w_i}{S}\right) + Z \quad \Rightarrow \quad W_q = [57;\ -57;\ 38;\ -128;\ 127;\ -37]$$

Toutes les valeurs sont bien comprises dans l'intervalle Int8.

Étape 5 – Vérification par déquantification : pour la première valeur,

$$r \approx 0{,}0059608 \times (57 - 18) = 0{,}0059608 \times 39 \approx 0{,}2325$$

soit une erreur de 0,0025 par rapport à la valeur d'origine 0,23.

Valeur d'origine	Valeur quantifiée	Valeur déquantifiée	Erreur absolue
0,23	57	0,2325	0,0025
−0,45	−57	−0,4471	0,0029
0,12	38	0,1192	0,0008
−0,87	−128	−0,8703	0,0003
0,65	127	0,6497	0,0003
−0,33	−37	−0,3278	0,0022

Table 2.1 – Exemple de quantification Int8 d'un tenseur de six poids ($S \approx 0{,}0059608$, $Z = 18$).

Conclusion de l'exemple : l'erreur d'arrondi est toujours inférieure à $S/2 \approx 0{,}003$. Elle est généralement acceptable pour les réseaux de neurones : la perte de précision globale du modèle reste le plus souvent de l'ordre de 1 à 2 points [12, 13].

2.3.2.3 Avantages

— **Réduction de taille** : division par quatre de la taille des poids. Pour un modèle de 20 Mo, cela représente une économie de 15 Mo.

— **Amélioration des performances** : de nombreux microcontrôleurs exécutent les opérations entières bien plus rapidement que les opérations flottantes. Les opérations Int8 sont particulièrement efficaces sur les architectures disposant d'instructions SIMD.

— **Réduction de la consommation énergétique** : une addition 8 bits entière consomme environ 30 fois moins d'énergie qu'une addition flottante 32 bits, et une multiplication environ 18 fois moins [11].

— **Compatibilité avec le matériel** : de nombreux accélérateurs d'IA (NPU) sont optimisés pour les calculs Int8.

2.3.2.4 Méthodes de quantification

Quantification post-entraînement (PTQ) : convertit un modèle déjà entraîné. Rapide et simple, elle suffit dans la majorité des cas. La perte de précision est généralement inférieure à 1–2 % pour la classification [13].

Quantification consciente de l'entraînement (QAT) : intègre la quantification dès la phase d'entraînement via une *fake quantization*. Elle permet d'atteindre des quantifications plus agressives (Int4, binaire) avec une meilleure préservation de la précision [12].

Listing 2.1 – Quantification post-entraînement Int8 avec TensorFlow Lite.

```
import tensorflow as tf

converter = tf.lite.TFLiteConverter.from_saved_model(saved_model_dir)
converter.optimizations = [tf.lite.Optimize.DEFAULT]
# Jeu de données représentatif pour calibrer scale et zero_point
converter.representative_dataset = generateur_donnees_representatives
converter.target_spec.supported_ops = [tf.lite.OpsSet.TFLITE_BUILTINS_INT8]
converter.inference_input_type = tf.int8
converter.inference_output_type = tf.int8
quantized_model = converter.convert()
open("modele_int8.tflite", "wb").write(quantized_model)
```

2.3.2.5 Limites et défis

— **Perte de précision** : généralement faible pour la classification, elle peut être significative pour la détection d'objets ou les opérations sensibles.

— **Gestion des valeurs aberrantes** (*outliers*) : les valeurs extrêmes élargissent la plage et dégradent la résolution des autres.

— **Solutions** : quantification par canal, écrêtage (*clipping*), utilisation de la QAT.

— **Compatibilité** : certains opérateurs ne sont pas disponibles en Int8 dans les moteurs d'inférence embarqués.

2.3.2.6 Ordres de grandeur

Modèle	Taille Float32	Taille Int8	Perte de précision	Gain de vitesse
MobileNetV1	16,9 Mo	4,3 Mo	−0,8 %	×3,2
MobileNetV2	14,0 Mo	3,5 Mo	−1,2 %	×4,1
InceptionV3	92,0 Mo	23,0 Mo	−1,5 %	×2,8

Table 2.2 – Comparaison indicative des performances avant et après quantification Int8 (ordres de grandeur ; voir [13, 12]).

2.3.3 L'élagage (*pruning*)

2.3.3.1 Définition et motivation

L'élagage est une technique de compression structurelle qui supprime les connexions non essentielles [9]. Les réseaux entraînés contiennent de nombreuses connexions dont les poids sont proches de zéro et qui contribuent peu à la décision finale.

Origine biologique : l'élagage s'inspire des mécanismes de plasticité synaptique, dans lesquels les connexions les moins utilisées sont éliminées.

2.3.3.2 Méthodologies

Élagage non structuré : suppression des poids individuellement en fonction de leur magnitude. Le taux de compression peut être très élevé (90–95 %), mais il nécessite des formats de stockage creux spécifiques pour se traduire par un gain réel.

Élagage structuré : suppression de blocs entiers (filtres, neurones, canaux). Il réduit effectivement le nombre d'opérations et reste compatible avec le matériel existant.

Algorithme typique : (1) entraînement initial ; (2) tri des poids par magnitude ; (3) sélection des N % de poids les plus petits ; (4) mise à zéro ; (5) réentraînement court (*fine-tuning*).

Listing 2.2 – Élagage fondé sur un critère d'importance (approximation de Taylor).

```python
import numpy as np

def taylor_pruning(weights, gradients, pruning_rate=0.5):
    importance = np.abs(weights * gradients)
    threshold = np.percentile(importance, pruning_rate * 100)
    mask = importance > threshold
    return weights * mask, mask
```

2.3.3.3 Processus complet

Le processus enchaîne pré-entraînement, élagage (structuré ou non), *fine-tuning* et itération progressive. **Stratégie recommandée :** plusieurs itérations avec des taux modérés (par exemple 20 % à chaque itération) plutôt qu'une seule itération avec un taux élevé.

2.3.3.4 Résultats expérimentaux

Taux d'élagage	Précision	Perte	Réduction des paramètres
20 %	70,9 %	−0,3 %	20 %
40 %	70,1 %	−1,1 %	40 %
60 %	68,5 %	−2,7 %	60 %
80 %	65,2 %	−6,0 %	80 %

Table 2.3 – Impact indicatif de l'élagage sur la précision de MobileNetV1 (ImageNet).

2.3.4 La distillation (*knowledge distillation*)

2.3.4.1 Concept

La distillation transfère les connaissances d'un modèle complexe (le *professeur*) vers un modèle plus petit (l'*élève*), permettant de créer des modèles compacts tout en conservant une grande partie des performances [10].

Analogie pédagogique : un expert transmet non seulement les réponses finales, mais également son raisonnement à un élève.

2.3.4.2 Principe théorique

Le transfert de connaissances peut porter sur les sorties (*output knowledge*), sur les représentations intermédiaires (*intermediate knowledge*) ou sur les relations entre exemples (*relation knowledge*). La fonction de perte combine deux termes :

$$L_{\text{total}} = \alpha\, L_{\text{classification}} + \beta\, L_{\text{distillation}} \tag{2.2}$$

Une **température** $T > 1$ est utilisée pour adoucir les distributions de probabilités produites par le professeur [10] :

$$P_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)} \tag{2.3}$$

2.3.4.3 Méthodes

— **Distillation standard** : l'élève apprend à reproduire les sorties du professeur.

— **Distillation par représentations intermédiaires** : l'élève apprend les représentations de plusieurs couches internes.

— **Distillation mutuelle** : plusieurs élèves s'enseignent mutuellement.

— **Auto-distillation** : un modèle enseigne à une version plus petite de lui-même.

2.3.4.4 Résultats

Professeur	Élève	Param. P	Param. E	Précision P	Précision E
ResNet-152	ResNet-18	60,2 M	11,7 M	78,3 %	74,8 %
ResNet-152	MobileNet	60,2 M	4,2 M	78,3 %	71,3 %

Table 2.4 – Comparaison indicative des performances (top-1 ImageNet) avant et après distillation ; M = millions de paramètres.

2.3.5 Combinaison des techniques

Les techniques sont complémentaires et sont généralement appliquées dans l'ordre suivant [9] : (1) distillation pour réduire la taille de l'architecture ; (2) élagage pour réduire la complexité ; (3) quantification pour réduire la précision numérique ; avec un *fine-tuning* après chaque étape.

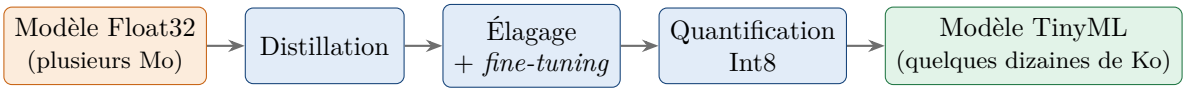


Figure 2.2 – Enchaînement typique des techniques de compression.

Méthode	Réduction de taille	Perte de précision	Gain de vitesse
Quantification seule	×4	< 1 %	×2 à 4
Élagage seul	×5 à 10	2 à 5 %	×1 à 2
Distillation seule	×10 à 50	3 à 8 %	×1 à 3
Combinaison	×100 à 300	5 à 15 %	×5 à 15

Table 2.5 – Efficacité comparée des méthodes de compression (ordres de grandeur).

2.3.6 Techniques avancées

— **Factorisation de matrices** : décomposition des matrices de poids en produits de matrices de plus petites dimensions (SVD, rang faible).

— **Binarisation** : représentation des poids et activations par des valeurs binaires (-1 ou $+1$). La réduction de taille est extrême ($\times 32$) mais la perte de précision importante.

— **Réseaux creux** : utilisation de représentations creuses pour les matrices de poids (formats COO, CSR).

— **Recherche automatique d'architecture (NAS)** : conception automatique de réseaux adaptés à l'embarqué, comme MobileNet, EfficientNet ou MCUNet [14].

2.3.7 Métriques d'évaluation

— **Taux de compression** : $\text{Taille}_{\text{originale}}/\text{Taille}_{\text{compressée}}$.

— **Métriques de performance** : précision, latence, consommation énergétique, mémoire RAM et flash.

— **Bancs d'essai** : ImageNet, CIFAR-10/100 et, pour l'embarqué, MLPerf Tiny [4].

2.3.8 Études de cas

— **Détection de mots-clés** : un modèle CNN de l'ordre du mégaoctet peut être ramené à une centaine de kilooctets, avec une précision supérieure à 90 % et une latence de quelques dizaines de millisecondes sur microcontrôleur [4].

— **Classification d'images** : des réseaux de type MobileNet ou MCUNet, quantifiés en Int8, tiennent dans moins de 1 Mo et s'exécutent sur des microcontrôleurs haut de gamme [14].

— **Séries temporelles** (vibrations, capteurs inertiels) : de petits réseaux denses ou auto-encodeurs de quelques dizaines de kilooctets suffisent pour la détection d'anomalies, avec une latence de quelques millisecondes sur ESP32 ; c'est le cas de notre exemple fil rouge.

Partie 3

Matériel et outils de développement

Les techniques de compression ne suffisent pas à elles seules : leur efficacité dépend du matériel sur lequel le modèle est exécuté et des outils logiciels qui assurent la conversion et le déploiement. Cette partie présente trois architectures représentatives de l'évolution des microcontrôleurs pour le TinyML, puis la chaîne d'outils logiciels associée.

3.1 Architectures matérielles

3.1.1 ESP32-S3 (Espressif)

— **Cœur** : Xtensa LX7 double cœur à 240 MHz ;

— **Mémoire** : 512 Ko de SRAM, jusqu'à 8 Mo de PSRAM externe ;

— **Accélération** : extensions vectorielles (SIMD) pour les calculs 8/16 bits ;

— **Connectivité** : Wi-Fi et Bluetooth LE intégrés [22].

3.1.2 ESP32-P4 (Espressif)

— **Architecture** : RISC-V double cœur jusqu'à 400 MHz ;

— **Mémoire** : jusqu'à 32 Mo de PSRAM ;

— **Accélération** : extensions dédiées à l'IA, traitement d'image matériel ;

— **Sortie** : 2024 [21].

3.1.3 STM32N6 (STMicroelectronics)

— **Cœur** : Arm Cortex-M55 à 800 MHz ;

— **NPU** : accélérateur neuronal Neural-ART à 600 GOPS ;

— **Efficacité** : environ 3 TOPS/W ;

— **Mémoire** : 4,2 Mo de SRAM intégrée [24].

Critère	ESP32-S3	ESP32-P4	STM32N6
Cœur	Xtensa LX7, 2 cœurs, 240 MHz	RISC-V, 2 cœurs, 400 MHz	Cortex-M55, 800 MHz
SRAM / PSRAM	512 Ko / jusqu'à 8 Mo	768 Ko / jusqu'à 32 Mo	4,2 Mo SRAM
Accélération IA	Instructions SIMD	Extensions IA, traitement d'image	NPU Neural-ART (600 GOPS)
Connectivité	Wi-Fi, BLE	Aucune radio intégrée	Aucune radio intégrée
Usage typique	Audio, capteurs, petite vision	Vision, interface homme-machine	Vision temps réel, modèles plus lourds

Table 3.1 – Comparaison des architectures matérielles pour le TinyML.

Ces trois architectures illustrent l'évolution des microcontrôleurs pour le TinyML, avec une complexité et des performances croissantes : des instructions vectorielles d'un MCU généraliste jusqu'à l'intégration d'un véritable accélérateur neuronal. Pour notre cas d'étude (détection d'anomalies vibratoires), un ESP32 ou un ESP32-S3 est largement suffisant ; le Wi-Fi intégré permet en outre de transmettre les alertes.

3.2 Chaîne d'outils logiciels

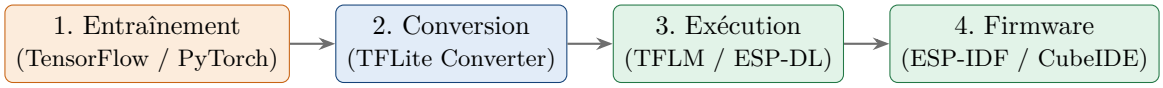


Figure 3.1 – Chaîne d'outils logiciels pour le développement TinyML.

3.2.1 TensorFlow Lite for Microcontrollers (TFLM)

TFLM est le moteur d'inférence de référence pour le TinyML [6, 30] :

- interpréteur C++ minimaliste, fonctionnant sans système d'exploitation ni allocation dynamique de mémoire ;
- empreinte de quelques dizaines de kilooctets seulement ;
- modèle intégré sous forme de tableau C dans le firmware ;
- *tensor arena* : zone de mémoire statique, réservée à la compilation, qui contient les activations pendant l'inférence.

La conversion d'un fichier `.tflite` en tableau C se fait en une ligne de commande, puis le modèle est chargé par l'interpréteur (listing 3.1).

Listing 3.1 – Initialisation de l'interpréteur TFLM sur microcontrôleur.

```
// Généré par : xxd -i modele_int8.tflite > modele_data.cc
#include "modele_data.h"
#include "tensorflow/lite/micro/micro_interpreter.h"
#include "tensorflow/lite/micro/micro_mutable_op_resolver.h"

constexpr int kTensorArenaSize = 20 * 1024;      // 20 Ko de SRAM
alignas(16) static uint8_t tensor_arena[kTensorArenaSize];

const tflite::Model* model = tflite::GetModel(modele_data);
static tflite::MicroMutableOpResolver<3> resolver;
resolver.AddFullyConnected();
resolver.AddRelu();
resolver.AddSoftmax();

static tflite::MicroInterpreter interpreter(model, resolver,
                                            tensor_arena, kTensorArenaSize);
interpreter.AllocateTensors();
```

3.2.2 Edge Impulse

Edge Impulse est une plateforme en ligne couvrant l'ensemble du pipeline, de la collecte des données au déploiement [27]. Elle permet d'enregistrer des données depuis la carte, de concevoir le prétraitement (FFT, MFCC) et le modèle, d'évaluer ses performances estimées sur le microcontrôleur cible, puis de générer automatiquement une bibliothèque C++ ou Arduino prête à être compilée.

3.2.3 Outils spécifiques aux fabricants

— **ESP-DL** (Espressif) : bibliothèque optimisée pour les ESP32, qui exploite leurs instructions vectorielles [28] ;

— **STM32Cube.AI** (STMicroelectronics) : outil de conversion et d'optimisation des modèles pour les STM32 [35] ;

— **NXP eIQ** : environnement de développement pour les microcontrôleurs et processeurs NXP [33].

Partie 4

Bénéfices et domaines d'application

4.1 Les bénéfices de l'AIoT et du TinyML

L'association de l'intelligence artificielle et de l'Internet des objets, appelée AIoT, permet aux équipements connectés d'analyser les données qu'ils collectent afin de reconnaître des situations ou de détecter des anomalies. Le TinyML correspond plus précisément à l'exécution de modèles d'apprentissage automatique sur des dispositifs disposant de ressources limitées, notamment des microcontrôleurs. Cette approche déplace une partie de l'intelligence au plus près des capteurs, plutôt que de transmettre systématiquement les données à un serveur distant. Elle présente plusieurs avantages en matière de réactivité, de confidentialité et de consommation énergétique [16].

4.1.1 Une meilleure réactivité et une moindre dépendance au réseau

Dans une architecture reposant sur le cloud, les données doivent être transmises à un serveur avant d'être analysées. Le délai de réponse dépend donc non seulement du calcul, mais aussi de la connexion réseau et des échanges avec le serveur. Avec le TinyML, l'analyse peut être réalisée directement dans l'objet connecté, ce qui évite cet aller-retour pour les décisions prises localement.

Cette organisation est particulièrement intéressante lorsqu'un système doit réagir rapidement, par exemple pour reconnaître un événement sonore ou détecter un comportement inhabituel d'une machine. Elle permet également de maintenir certaines fonctions d'analyse lorsque la connexion Internet est indisponible, à condition que ces fonctions aient été conçues pour fonctionner de manière autonome.

Il serait toutefois incorrect de parler de « latence zéro ». L'acquisition des données, leur préparation et l'exécution du modèle nécessitent toujours du temps. Le bénéfice correspond à une réduction de la latence et de sa dépendance au réseau, et non à sa disparition.

4.1.2 Une meilleure maîtrise de la confidentialité des données

Les objets connectés peuvent collecter des informations sensibles, comme des enregistrements audio, des données de mouvement ou des mesures physiologiques. Lorsque ces données sont systématiquement envoyées vers un serveur, leur traitement implique davantage de transferts et de stockage à distance. L'inférence locale permet de limiter ces échanges en analysant les informations directement sur l'appareil.

Par exemple, un dispositif de reconnaissance sonore peut analyser un signal audio localement et ne transmettre que le résultat de la détection. Selon l'architecture retenue, il devient ainsi possible de communiquer une alerte plutôt que l'ensemble de l'enregistrement. Le TinyML contribue donc à réduire la quantité de données brutes qui quittent l'objet connecté.

Cet avantage ne signifie pas que la confidentialité est automatiquement garantie. Certains systèmes peuvent encore transmettre des données pour l'entraînement, le suivi ou d'autres services. Il faut donc distinguer la possibilité de traiter les données localement du choix réel de ne pas les transmettre.

4.1.3 Une réduction possible de la consommation énergétique

L'autonomie constitue une contrainte importante pour les objets connectés alimentés par batterie. Les communications sans fil consomment de l'énergie, notamment lorsque l'appareil transmet fréquemment des mesures. En analysant les données localement et en ne communiquant que les résultats utiles, le TinyML peut réduire le recours à la transmission radio [2].

Cependant, le calcul local consomme lui aussi de l'énergie. Le gain final dépend donc de plusieurs paramètres : la complexité du modèle, la fréquence des analyses, la consommation des capteurs, les modes de veille et la fréquence des communications. Une analyse locale bien dimensionnée peut améliorer l'autonomie, mais cet avantage doit être mesuré sur le dispositif complet plutôt que supposé à partir du seul modèle d'IA [4].

Bénéfice	Ce que le TinyML apporte	Nuance à garder en tête
Réactivité	Décision locale, sans aller-retour réseau	La latence est réduite, pas supprimée
Fonctionnement hors ligne	Analyse maintenue sans connexion	Seulement pour les fonctions conçues pour cela
Confidentialité	Les données brutes peuvent rester dans l'objet	Dépend des choix réels de transmission
Énergie	Moins de transmissions radio	Le calcul local consomme aussi
Coût	Moins de données envoyées et stockées dans le cloud	Coût de développement et de mise à jour

Table 4.1 – Synthèse des bénéfices du TinyML et de leurs limites.

4.2 Domaines d'application et exemples d'utilisation

4.2.1 Industrie : surveillance des machines et maintenance

Dans l'industrie, l'apprentissage automatique embarqué peut servir à surveiller le fonctionnement des machines et à détecter des signes de dégradation. La maintenance prédictive fait notamment partie des applications de l'IA en périphérie : l'objectif est d'identifier des défauts avant qu'ils n'entraînent des problèmes plus importants [18].

Pour illustrer ce principe, notre projet retient le cas d'un moteur surveillé par un capteur de vibrations. Le scénario étudié consiste à analyser les mesures sur un microcontrôleur, puis à déclencher une alerte lorsqu'un comportement inhabituel est détecté.

L'intérêt recherché est de fournir une information utile au personnel de maintenance sans transmettre continuellement toutes les mesures à un serveur. Il convient néanmoins de distinguer la détection d'anomalies de la prédiction d'une panne : repérer un fonctionnement inhabituel ne permet pas nécessairement de déterminer sa cause ou le temps restant avant une défaillance. La détection constitue un premier niveau de surveillance ; la prédiction demande une analyse et une validation supplémentaires.

4.2.2 Maison connectée : reconnaissance de mots-clés et de sons

La reconnaissance de mots-clés est une application représentative du TinyML ; elle fait d'ailleurs partie des tâches de référence du banc d'essai MLPerf Tiny [4]. Un modèle compact peut être chargé d'identifier un nombre limité de commandes vocales sur un appareil disposant de peu de mémoire et de puissance de calcul. Cette tâche ciblée se distingue d'une conversation complète avec un assistant fondé sur un grand modèle de langage.

L'analyse locale présente ici un double intérêt : réduire les échanges avec un serveur et permettre une réaction rapide. Elle peut aussi limiter la transmission de données audio lorsque le système est conçu pour ne communiquer que les résultats nécessaires.

La reconnaissance reste cependant sensible aux conditions d'utilisation. Le bruit ambiant peut perturber la détection et provoquer des erreurs. Pour obtenir un système robuste, il faut donc entraîner et tester le modèle avec des données représentatives des environnements dans lesquels l'objet sera réellement utilisé.

Partie 5

Limites, défis et perspectives d'avenir

5.1 Des ressources matérielles limitées

La principale difficulté du TinyML consiste à faire fonctionner un modèle utile sur un appareil disposant de ressources réduites. La mémoire vive, l'espace de stockage, la puissance de calcul et l'énergie disponible limitent la complexité des modèles déployables. L'optimisation doit ainsi prendre en compte plusieurs critères simultanément : la taille du modèle, sa précision, son temps d'exécution et sa consommation [5].

Les outils logiciels imposent également des contraintes. TensorFlow Lite for Microcontrollers, par exemple, prend en charge un ensemble limité d'opérations et nécessite une gestion adaptée de la mémoire [6]. Un modèle conçu sur ordinateur ne peut donc pas toujours être transféré directement sur un microcontrôleur sans modification.

Ces contraintes conduisent à rechercher un compromis : le modèle le plus complexe n'est pas nécessairement le plus pertinent si un modèle plus simple répond au besoin tout en consommant moins de ressources.

5.2 Une fiabilité dépendante des données et du contexte

Les performances d'un modèle dépendent des données utilisées pour son entraînement et de leur représentativité. Dans les applications vocales, le bruit peut dégrader la reconnaissance. Dans les dispositifs portables, les mouvements, les différences entre utilisateurs ou les variations de positionnement des capteurs peuvent modifier les signaux analysés.

Un modèle peut donc donner de bons résultats pendant les essais puis devenir moins fiable lorsque les conditions changent. Il est nécessaire de le tester dans des situations variées et de suivre ses performances après le déploiement. Cette exigence est particulièrement importante lorsque les erreurs peuvent avoir des conséquences sur la surveillance d'un équipement ou sur l'interprétation de données de santé.

5.3 L'entraînement et l'évolution des modèles

Dans de nombreux systèmes TinyML, l'entraînement est effectué sur un ordinateur ou une infrastructure distante, tandis que le microcontrôleur réalise uniquement l'inférence, c'est-à-dire l'utilisation du modèle déjà entraîné. Cette séparation s'explique par les besoins généralement plus élevés de l'entraînement en mémoire et en puissance de calcul. TensorFlow Lite for Microcontrollers ne prend d'ailleurs pas en charge l'entraînement sur l'appareil [30].

Cette organisation soulève la question de l'évolution du système. Lorsque les données ou les conditions d'utilisation changent, une nouvelle version du modèle peut devenir nécessaire ; elle doit alors être redéployée, par exemple par une mise à jour du firmware à distance (OTA).

L'apprentissage directement sur l'appareil constitue une piste d'évolution, des travaux ayant montré qu'il est possible d'entraîner partiellement un réseau avec moins de 256 Ko de mémoire [15]. Il n'est cependant pas encore une solution générale pour tous les microcontrôleurs : il reste limité par les ressources disponibles et par la complexité supplémentaire qu'il introduit.

5.4 La sécurité des modèles embarqués

Placer un modèle dans un objet physique l'expose à de nouvelles menaces : un attaquant ayant accès à l'appareil peut tenter d'extraire le modèle de la mémoire flash, de le modifier ou de fournir au capteur des entrées conçues pour le tromper. La protection de la mémoire (chiffrement de la flash, démarrage sécurisé) et la sécurisation des mises à jour OTA, que proposent par exemple les ESP32 [20], font donc partie intégrante de la conception d'un système AIoT.

5.5 Vers des architectures complémentaires entre l'objet et le cloud

Le développement du TinyML ne signifie pas nécessairement la disparition du cloud. Les deux approches peuvent remplir des rôles complémentaires : l'objet exécute les analyses locales compatibles avec ses ressources, tandis qu'une infrastructure plus puissante peut prendre en charge l'entraînement et les traitements plus exigeants.

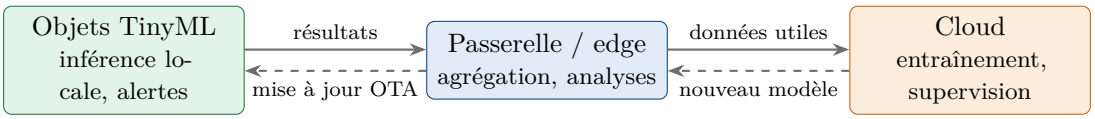


Figure 5.1 – Répartition des traitements dans une architecture AIoT hybride.

Les perspectives reposent notamment sur l'amélioration de l'efficacité des modèles et de leur adaptation aux contraintes matérielles. Les travaux récents sur la quantification montrent que la recherche vise à réduire les besoins en mémoire et en énergie tout en préservant des performances adaptées à l'application [8]. Parallèlement, les microcontrôleurs intègrent de plus en plus d'accélérateurs neuronaux dédiés, comme le montre le STM32N6 présenté dans la partie 3.

L'enjeu n'est donc pas de placer toute l'intelligence dans chaque capteur, mais de répartir les traitements de façon pertinente selon les besoins de réactivité, d'autonomie et de confidentialité.

Conclusion générale

Ce rapport a montré comment l'intelligence artificielle peut quitter les centres de données pour s'installer au cœur des objets connectés. Partant de l'architecture classique de l'IoT, où l'objet se contente de mesurer et de transmettre, nous avons vu que l'AIoT donne aux systèmes connectés la capacité d'interpréter leurs données, et que le TinyML pousse cette logique jusqu'aux microcontrôleurs les plus modestes.

Cette évolution repose sur une séparation nette entre l'entraînement, réalisé sur des machines puissantes, et l'inférence, seule exécutée dans l'objet. Elle n'est possible que grâce à un pipeline de développement rigoureux et à des techniques de compression (quantification, élagage, distillation) capables de réduire la taille d'un modèle d'un ou deux ordres de grandeur, ainsi qu'à des microcontrôleurs et des outils logiciels de plus en plus adaptés.

Les bénéfices sont réels : réactivité accrue, fonctionnement en l'absence de réseau, réduction des données transmises et donc meilleure maîtrise de la confidentialité et, souvent, de la consommation énergétique. Ils doivent cependant être nuancés : la latence n'est pas nulle, le calcul local consomme lui aussi de l'énergie, la fiabilité dépend fortement des données d'entraînement et l'évolution des modèles déployés reste délicate.

Le TinyML ne remplace donc pas le cloud : il le complète. L'avenir de l'AIoT se dessine comme une intelligence distribuée, dans laquelle chaque niveau (objet, passerelle, cloud) exécute les traitements qui lui conviennent le mieux. Notre exemple de détection d'anomalies vibratoires sur ESP32 illustre bien cette répartition : l'objet surveille et alerte en temps réel, tandis que le cloud supervise, archive et améliore le modèle au fil du temps.

Bibliographie et webographie

Ouvrages de référence

[1] V. Sze, Y.-H. Chen, T.-J. Yang et J. S. Emer. *Efficient Processing of Deep Neural Networks*. Synthesis Lectures on Computer Architecture. Morgan & Claypool, 2020. DOI : 10.2200/S01004ED1V01Y202004CAC050.

[2] P. Warden et D. Situnayake. *TinyML : Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. Sebastopol, CA : O'Reilly Media, 2019. ISBN : 978-1-492-05204-3.

Articles scientifiques

[3] L. Atzori, A. Iera et G. Morabito. « The Internet of Things : A survey ». In : *Computer Networks* 54.15 (2010), p. 2787-2805. DOI : 10.1016/j.comnet.2010.05.010.

[4] C. Banbury et al. « MLPerf Tiny Benchmark ». In : *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*. 2021. arXiv : 2106.07597.

[5] C. R. Banbury et al. *Benchmarking TinyML Systems : Challenges and Direction*. 2020. arXiv : 2003.04821.

[6] R. David et al. « TensorFlow Lite Micro : Embedded Machine Learning for TinyML Systems ». In : *Proceedings of Machine Learning and Systems (MLSys)*. T. 3. 2021. arXiv : 2010.08678.

[7] A. Al-Fuqaha et al. « Internet of Things : A Survey on Enabling Technologies, Protocols, and Applications ». In : *IEEE Communications Surveys & Tutorials* 17.4 (2015), p. 2347-2376. DOI : 10.1109/COMST.2015.2444095.

[8] A. Gholami et al. « A Survey of Quantization Methods for Efficient Neural Network Inference ». In : *Low-Power Computer Vision*. Chapman et Hall/CRC, 2022. arXiv : 2103.13630.

[9] S. Han, H. Mao et W. J. Dally. « Deep Compression : Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding ». In : *International Conference on Learning Representations (ICLR)*. 2016. arXiv : 1510.00149.

[10] G. Hinton, O. Vinyals et J. Dean. *Distilling the Knowledge in a Neural Network*. NIPS Deep Learning and Representation Learning Workshop. 2015. arXiv : 1503.02531.

[11] M. Horowitz. « Computing's energy problem (and what we can do about it) ». In : *IEEE International Solid-State Circuits Conference (ISSCC)*. 2014, p. 10-14. DOI : 10.1109/ISSCC.2014.6757323.

[12] B. Jacob et al. « Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference ». In : *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2018, p. 2704-2713. arXiv : 1712.05877.

[13] R. Krishnamoorthi. *Quantizing deep convolutional networks for efficient inference : A whitepaper*. 2018. arXiv : 1806.08342.

[14] J. Lin et al. « MCUNet : Tiny Deep Learning on IoT Devices ». In : *Advances in Neural Information Processing Systems (NeurIPS)*. T. 33. 2020. arXiv : 2007.10319.

[15] J. Lin et al. « On-Device Training Under 256KB Memory ». In : *Advances in Neural Information Processing Systems (NeurIPS)*. T. 35. 2022. arXiv : 2206.15472.

[16] P. P. Ray. « A review on TinyML : State-of-the-art and prospects ». In : *Journal of King Saud University – Computer and Information Sciences* 34.4 (2022), p. 1595-1623. DOI : 10.1016/j.jksuci.2021.11.019.

[17] W. Shi et al. « Edge Computing : Vision and Challenges ». In : *IEEE Internet of Things Journal* 3.5 (2016), p. 637-646. DOI : 10.1109/JIOT.2016.2579198.

[18] J. Zhang et D. Tao. « Empowering Things With Intelligence : A Survey of the Progress, Challenges, and Opportunities in Artificial Intelligence of Things ». In : *IEEE Internet of Things Journal* 8.10 (2021), p. 7789-7817. DOI : 10.1109/JIOT.2020.3039359.

Normes, rapports et documentation technique

[19] K. Ashton. « That 'Internet of Things' Thing ». In : *RFID Journal* (juin 2009).
[20] Espressif Systems. *ESP32 Series Datasheet*. Espressif Systems. URL : https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf (visité le 05/11/2026).
[21] Espressif Systems. *ESP32-P4 Series Datasheet*. Espressif Systems. URL : https://www.espressif.com/en/products/socs/esp32-p4 (visité le 05/11/2026).
[22] Espressif Systems. *ESP32-S3 Series Datasheet*. Espressif Systems. URL : https://www.espressif.com/sites/default/files/documentation/esp32-s3_datasheet_en.pdf (visité le 05/11/2026).
[23] IoT Analytics. *State of IoT 2024 : Number of connected IoT devices growing 13% to 18.8 billion globally*. 2024. URL : https://iot-analytics.com/number-connected-iot-devices/ (visité le 05/11/2026).
[24] STMicroelectronics. *STM32N6 series – Arm Cortex-M55 microcontrollers with Neural-ART Accelerator*. STMicroelectronics. URL : https://www.st.com/en/microcontrollers-microprocessors/stm32n6-series.html (visité le 05/11/2026).
[25] Union internationale des télécommunications. *Recommandation UIT-T Y.2060 : Présentation générale de l'Internet des objets*. Rapp. tech. Genève : UIT-T, 2012. URL : https://www.itu.int/rec/T-REC-Y.2060-201206-I.

Ressources en ligne et communautés

[26] EDGE AI FOUNDATION. *EDGE AI FOUNDATION (anciennement tinyML Foundation)*. URL : https://www.edgeaifoundation.org/ (visité le 05/11/2026).
[27] Edge Impulse. *Edge Impulse – Documentation*. URL : https://docs.edgeimpulse.com/ (visité le 05/11/2026).
[28] Espressif Systems. *ESP-DL : bibliothèque d'apprentissage profond pour ESP32*. URL : https://github.com/espressif/esp-dl (visité le 05/11/2026).
[29] Espressif Systems. *ESP-IDF Programming Guide*. URL : https://docs.espressif.com/projects/esp-idf/ (visité le 05/11/2026).
[30] Google / TensorFlow. *TensorFlow Lite for Microcontrollers (LiteRT for Microcontrollers)*. URL : https://ai.google.dev/edge/litert/microcontrollers/overview (visité le 05/11/2026).
[31] Google / TensorFlow. *TensorFlow Model Optimization Toolkit*. URL : https://www.tensorflow.org/model_optimization (visité le 05/11/2026).
[32] MLCommons. *MLPerf Inference : Tiny*. URL : https://mlcommons.org/benchmarks/inference-tiny/ (visité le 05/11/2026).
[33] NXP Semiconductors. *eIQ Machine Learning Software Development Environment*. URL : https://www.nxp.com/design/design-center/software/eiq-ai-development-environment:EIQ (visité le 05/11/2026).
[34] PyTorch. *Quantization – PyTorch documentation*. URL : https://pytorch.org/docs/stable/quantization.html (visité le 05/11/2026).
[35] STMicroelectronics. *X-CUBE-AI – AI expansion pack for STM32CubeMX*. URL : https://www.st.com/en/embedded-software/x-cube-ai.html (visité le 05/11/2026).